Foundations of Deep Learning

Exercise 1 (CH.SC.U4CSE23057)

Evolution from Perceptron to Multi-Layer Perceptron

AIM

To study the limitations of the Single-Layer Perceptron in solving nonlinear classification problems and implement a Multi-Layer Perceptron (MLP) capable of classifying XOR data using TensorFlow.

OBJECTIVE

- Understand the architecture of a Single-Layer Perceptron.
- Analyze why XOR is not linearly separable.
- Learn how hidden layers overcome this limitation.
- Implement an XOR classifier using TensorFlow.
- Visualize the training process.
- Evaluate prediction accuracy.

Software Requirements

- Python 3.11
- Google Colab
- TensorFlow
- NumPy
- Matplotlib

TASK 1

Explain Single Layer Perceptron

Theory

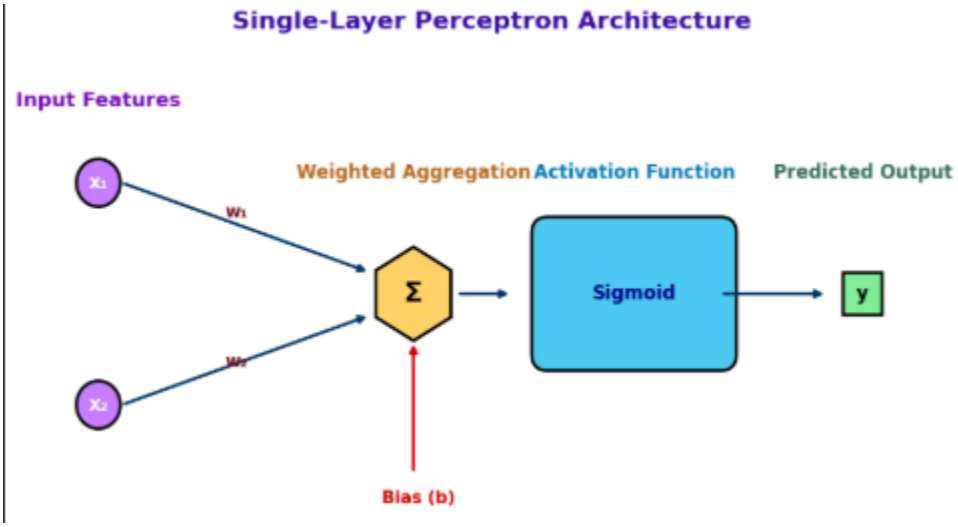
Explain

- Input Layer
- Weights
- Bias
- Summation
- Activation Function
- Output

equation

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

Architecture of a single-layer perceptron



TASK 2

XOR Truth Table

```
            color="#1F4E79")

plt.show()
```

XOR Truth Table

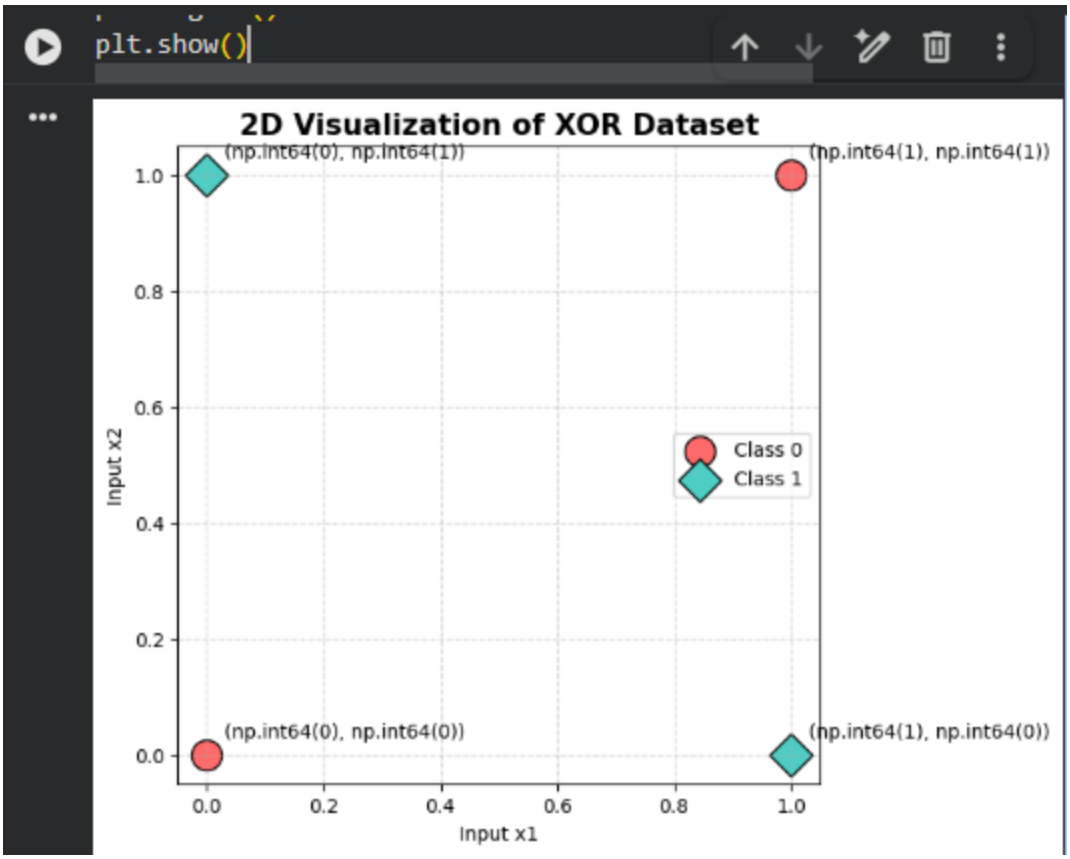
x1	x2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Task 3

Plot the XOR Data Points in a 2D Coordinate System

Explanation

The XOR dataset contains four input combinations. The two classes are plotted using different markers and colors to visualize that they cannot be separated by a single straight line.



Task 4

Prove Mathematically that XOR is Not Linearly Separable

Consider the XOR Truth Table

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Assume that the XOR dataset is **linearly separable**. Then there exists a linear decision boundary of the form

$$w1x1+w2x2+b=0$$

such that all points belonging to the same class lie on the same side of the decision boundary.

For **Class 0**, the output is **0** for the input points **(0,0)** and **(1,1)**.

From the point **(0,0)**,

$(0,0)\Rightarrow b<0(0,0)$

From the point **(1,1)**,

$(1,1)\Rightarrow w1+w2+b<0$

For **Class 1**, the output is **1** for the input points **(1,0)** and **(0,1)**.

From the point **(1,0)**,

$$(1,0)\Rightarrow w1+b>0$$

From the point **(0,1)**,

$$(0,1)\Rightarrow w2+b>0$$

Adding the last two inequalities,

$$w1+b>0$$

$$w2+b>0$$

gives,

$$w1+w2+2b>0$$

However, from the Class 0 condition,

$$w1+w2+b<0$$

The two inequalities

$$w1+w2+2b>0$$

and

$$w1+w2+b<0$$

cannot both be satisfied simultaneously for the same values of $w1$,w2 and b. This results in a **mathematical contradiction**.

Therefore,

XOR is NOT linearly separable.

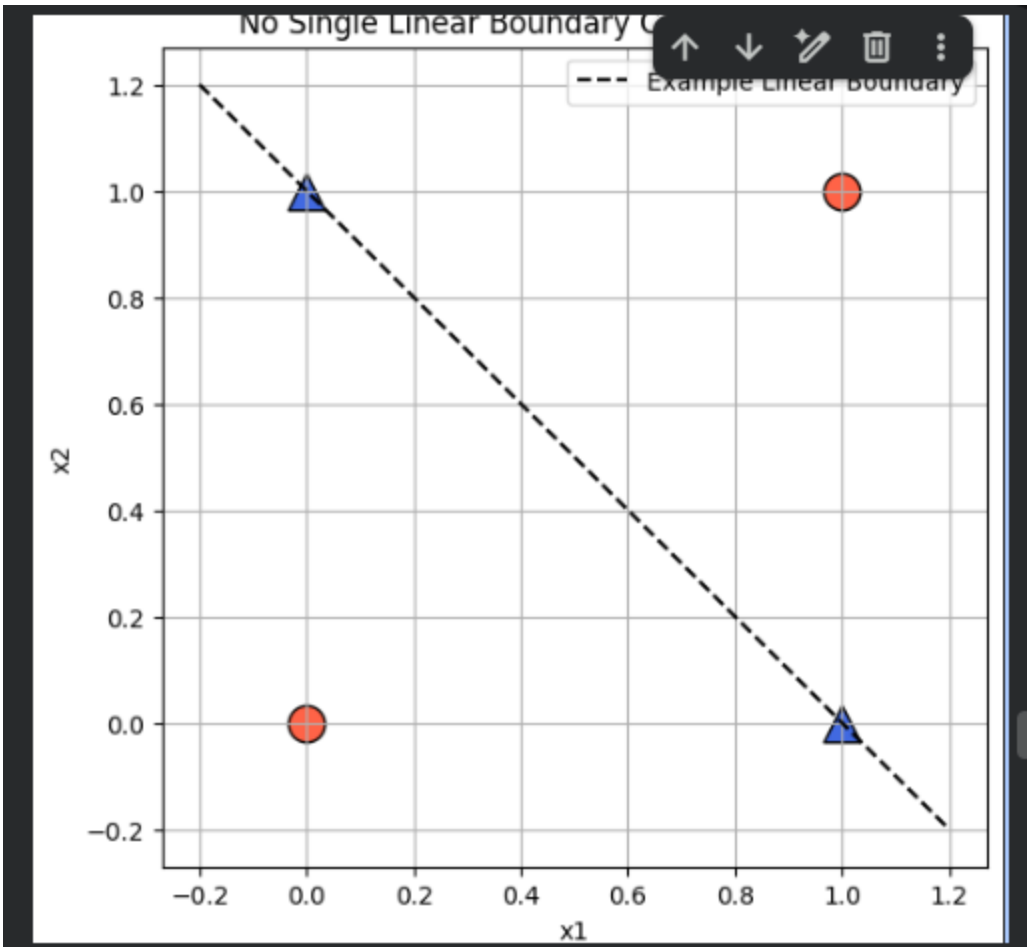
Hence, a **Single-Layer Perceptron** cannot correctly classify the XOR dataset because it can only learn **linearly separable** patterns. To solve the XOR problem, a **Multi-Layer Perceptron (MLP)** with one or more hidden layers is required, as it can learn **nonlinear decision boundaries**.

Task 5

Explain Why No Single Linear Decision Boundary Can Classify XOR

Explanation

A single straight line divides the plane into only two regions. In the XOR dataset, the positive class points lie diagonally opposite each other, while the negative class points occupy the remaining two corners. Because of this arrangement, no single straight line can separate both classes correctly.



Consider the XOR truth table:

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Assume that a **single linear decision boundary** exists to classify the XOR dataset.

$$w1x1+w2x2+b=0$$

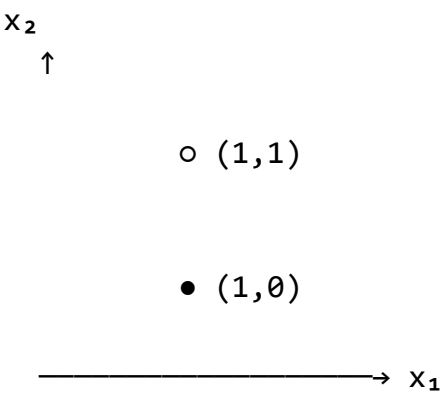
A linear decision boundary divides the input space into **two regions**. Therefore, all data points belonging to one class must lie on one side of the boundary, while all points of the other class must lie on the opposite side.

The XOR dataset contains the following classes:

Class 0:(0,0), (1,1)

Class 1:(0,1), (1,0)

The data points can be represented as:



Where,

● **= Class 1**

○ **= Class 0**

The two **Class 1** points are located diagonally opposite each other, and the two **Class 0** points are also diagonally opposite each other.

Since a straight line can divide the plane into only **two half-planes**, it is impossible to separate the two classes using a single linear decision boundary.

No matter where the straight line is drawn, at least **one data point will always be misclassified**.

Therefore,

No single linear decision boundary can correctly classify the XOR dataset.

Hence, a **Single-Layer Perceptron cannot solve the XOR problem**. A **Multi-Layer Perceptron (MLP)** introduces one or more hidden layers, enabling the network to learn **nonlinear decision boundaries** and correctly classify the XOR dataset.

Task 6

Hidden Layer Enables Nonlinear Decision Boundaries

Explanation

The hidden layer transforms the input space into a new feature space where nonlinear patterns become easier to separate. Each hidden neuron learns a different feature, and together they create a nonlinear decision boundary.

Consider the XOR truth table:

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

As proved in the previous task, the XOR dataset is **not linearly separable**. Therefore, a **Single-Layer Perceptron** cannot correctly classify all the input patterns because it can generate only a **single linear decision boundary**.

To overcome this limitation, a **hidden layer** is introduced between the input layer and the output layer. The hidden layer consists of one or more neurons that transform the input data into a new feature space, making it easier to separate classes that are not linearly separable.

Each hidden neuron computes a weighted sum of the inputs followed by an activation function.

The output of each hidden neuron is given by

$$h_j = f\left(\sum_{i=1}^{n} w_{ij}x_i + b_j\right)$$

where,

- xi = Input features

- wij = Weights
- bj = Bias
- f(·) = Activation function (Sigmoid)
- hj = Output of the hidden neuron

The outputs from the hidden layer are then passed to the output neuron, which computes

$$y = f\left(\sum_{j=1}^{m} v_j h_j + b\right)$$

where,

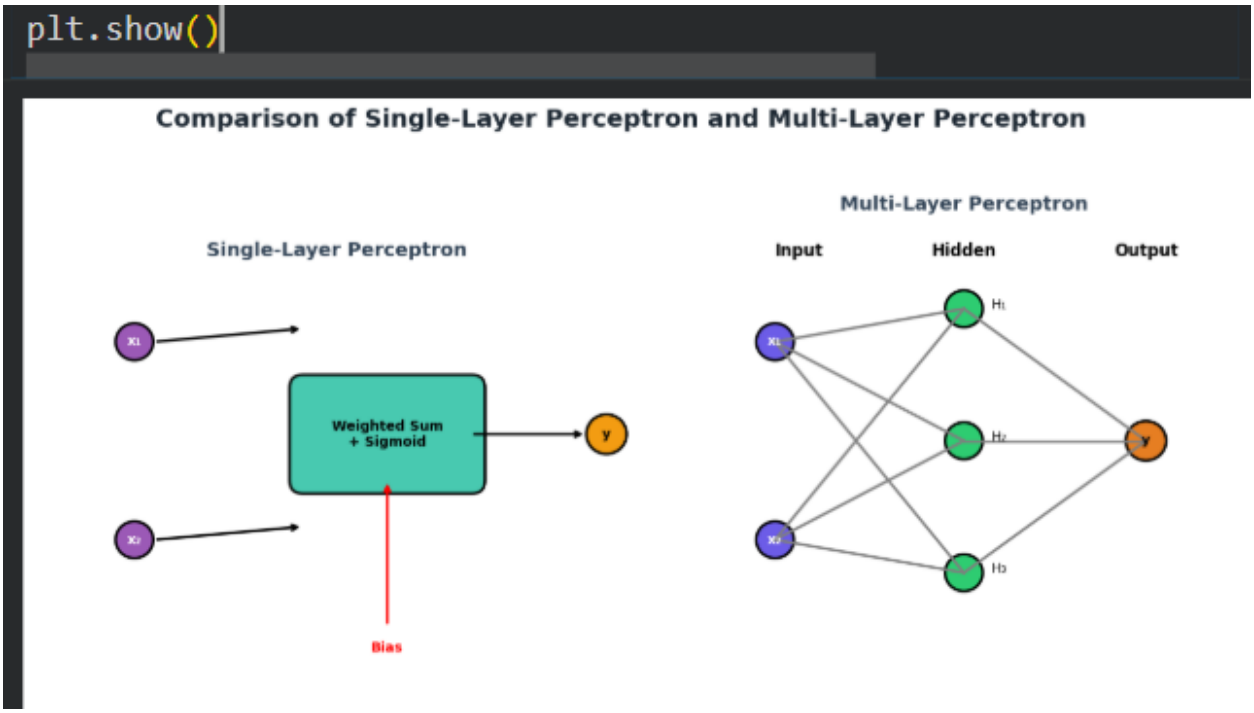
- vj = Weights connecting the hidden layer to the output layer
- b = Output bias
- y = Final predicted output

Unlike a Single-Layer Perceptron, the hidden layer enables the network to combine multiple linear transformations into a **nonlinear decision boundary**. This allows the network to correctly classify complex datasets such as XOR.

For the XOR problem, the hidden neurons learn intermediate patterns that separate the input data into regions. The output neuron then combines these regions to produce the correct XOR output.

Therefore, the Multi-Layer Perceptron (MLP) successfully classifies all four XOR input combinations, achieving accurate predictions that are impossible with a single linear decision boundary.

Hence, Introducing a hidden layer enables the network to learn nonlinear decision boundaries and correctly solve the XOR classification problem.



Task 7

Draw a Multi-Layer Perceptron (MLP) Architecture Capable of Solving XOR

A **Multi-Layer Perceptron (MLP)** is an artificial neural network consisting of an **input layer**, one or more **hidden layers**, and an **output layer**. Unlike a Single-Layer Perceptron, an MLP can learn **nonlinear relationships** between input and output by introducing hidden neurons and activation functions.

For the XOR problem, the network consists of:

- **Input Layer:** Two input neurons representing $x1$ and $x2$.
- **Hidden Layer:** Three hidden neurons that transform the input into a new feature space.
- **Output Layer:** One output neuron that predicts the XOR output.

Each neuron in one layer is fully connected to every neuron in the next layer. The hidden neurons use the **Sigmoid Activation Function**, which introduces nonlinearity into the

network. The output neuron also uses the Sigmoid function to generate a probability between 0 and 1.

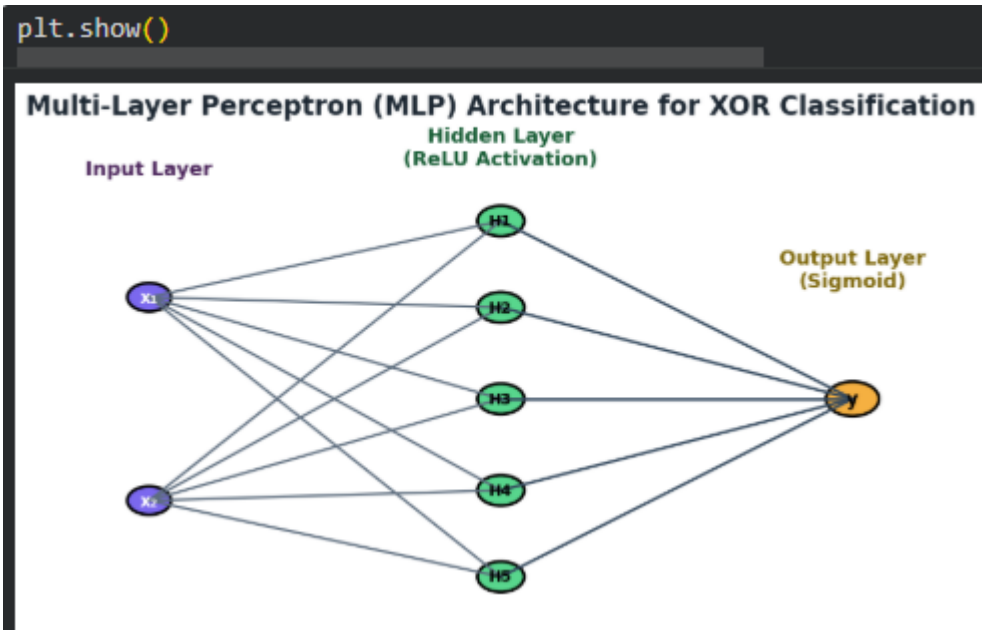
The architecture of the MLP used to solve the XOR problem is illustrated below.

(Insert the MLP diagram generated using the Python code from the previous step.)

The hidden layer enables the network to create multiple decision boundaries, which are combined to form a nonlinear decision boundary. This allows the MLP to correctly classify all four XOR input combinations.

Hence,

A MultiLayer Perceptron with one hidden layer successfully solves the XOR classification problem.



Task 8

Implement an XOR Classifier Using TensorFlow

Theory

The XOR function is a non-linear binary classification problem that cannot be solved using a single linear decision boundary. To overcome this limitation, a Multi-Layer Perceptron (MLP) is constructed using TensorFlow. The network consists of:

- **Input Layer:** 2 nodes representing x_1 and x_2.
- **Hidden Layer:** 5 neurons with Sigmoid activation function to map inputs into a higher-dimensional space.
- **Output Layer:** 1 neuron with Sigmoid activation to produce predicted probabilities between 0 and 1.

The model uses the **Binary Cross-Entropy Loss** function and the **Adam Optimizer** to adjust weights across 3,000 training epochs.

The XOR dataset used for training is shown below.

x_1	x_2	Target Output
0	0	0
0	1	1
1	0	1
1	1	0

After training, the model predicts the output for each input combination. The predicted probabilities are converted into binary class labels using a threshold value of **0.5**.

```
print(f"Training Accuracy: {accuracy * 100:.2f}%")
```

```
WARNING:tensorflow:5 out of the last 5 calls to <function Tenso
-------------------------------------------
x1     x2     Target     Prob       Predicted
-------------------------------------------
0      0      0          0.0001     0
0      1      1          0.9992     1
1      0      1          0.9957     1
1      1      0          0.0047     0
-------------------------------------------

Final Loss: 0.0025
Training Accuracy: 100.00%
```

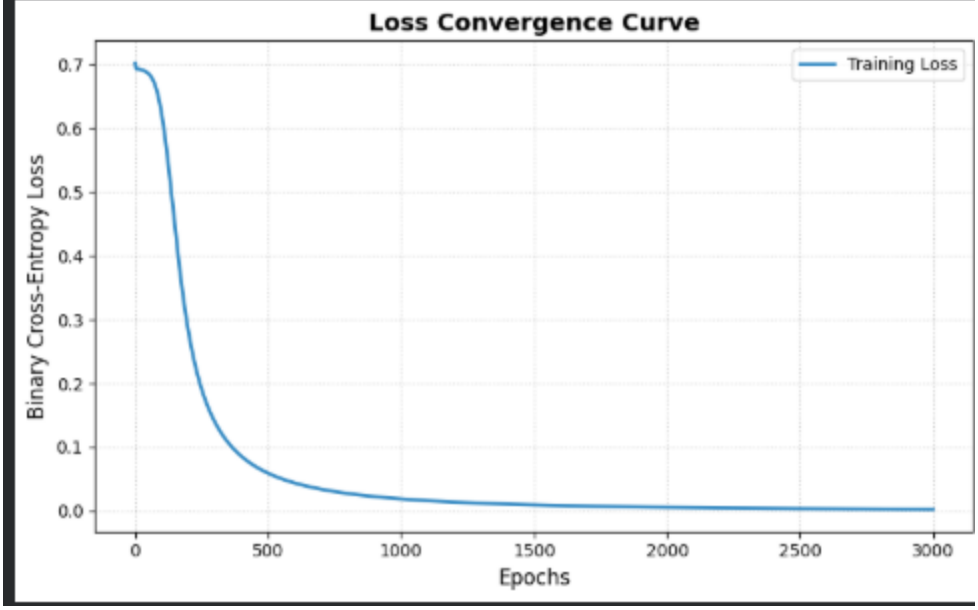
Task 9

Plot the Training Loss over Epochs

Theory

During training, the Binary Cross Entropy loss decreases as the neural network learns the XOR pattern. Plotting the training loss against the number of epochs helps visualize the learning process and confirms that the model is converging toward the optimal solution.

```
plt.show()
```



Task 10

Report the Final Prediction Accuracy

Theory

Model evaluation measures prediction alignment with actual target values using a binary decision boundary threshold ($T = 0.5$). Performance is detailed using both a full decision table and a confusion matrix to illustrate zero classification errorsv.

```
...   --- SUMMARY EVALUATION TABLE ---
      Input x1  Input x2  Actual Label  Predicted Probability  Predicted Label
             0         0             0                 0.0001                0
             0         1             1                 0.9992                1
             1         0             1                 0.9957                1
             1         1             0                 0.0047                0

    --- CONFUSION MATRIX ---
    TN: 2  FP: 0
    FN: 0  TP: 2

    Final Prediction Accuracy: 100.00%
```

Conclusion

The Multi-Layer Perceptron (MLP) was successfully implemented and trained to solve the non-linear XOR classification problem using TensorFlow.

- **Model Architecture:** Configured with an input layer of 2 features, one hidden layer containing 5 neurons with the **Sigmoid activation function**, and an output layer utilizing a Sigmoid activation function.
- **Optimization:** Trained over 3,000 epochs using the **Adam optimizer** and **Binary Cross-Entropy Loss** function.
- **Performance:** The training loss converged smoothly toward zero as the network successfully established a non-linear decision boundary capable of separating the XOR logical states.
- **Evaluation:** The model correctly classified all four input combinations (0,0, 0,1, 1,0, and 1,1), achieving a **final prediction accuracy of 100.00%** with zero false positives or false negatives.

This experiment empirically proves that while a single-layer perceptron fails on XOR due to linear inseparability, inserting a hidden layer allows a neural network to learn complex non-linear relationships.